

# Module 4

## GAN, Vision Transformers & Déploiement

Apprentissage adversarial — attention visuelle — mise en production

### Du génératif adversarial à la production

Générer — transformer par l'attention — déployer — projet fil rouge

$\min_G \max_D$

GAN

DCGAN, Pix2Pix, SRGAN

$QK^TV$

ViT & Swin

Self-attention



Déploiement

TorchScript, ONNX



Projet

Pipeline complet

Bassem Ben Hamed

[bassem.benhamed@enetcom.usf.tn](mailto:bassem.benhamed@enetcom.usf.tn)

ENETCOM — Université de Sfax

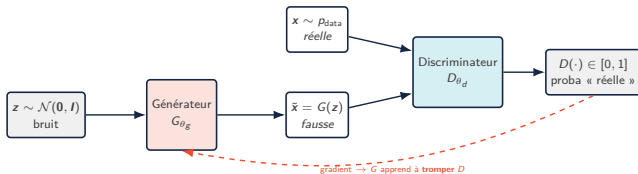
Présentation 04 — Module 4

1. GAN : le jeu minimax & ses variantes
2. Vision Transformers : ViT, Swin & SegFormer
3. Déploiement : TorchScript, ONNX & quantification
4. Projet fil rouge & synthèse

## GAN : le jeu minimax & ses variantes

---

# GAN — principe : le jeu minimax



## Objectif minimax (Goodfellow et al., 2014)

Deux réseaux s'affrontent dans un **jeu à somme nulle** —  $D$  maximise,  $G$  minimise la même valeur  $V$  :

$$\min_G \max_D V(D, G) = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)]}_{\text{(A) images réelles}} + \underbrace{\mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]}_{\text{(B) images générées}}$$

- **(A)** :  $D$  veut  $D(x) \rightarrow 1$  (image réelle)  $\Rightarrow$  maximise  $\log D(x)$  ;
- **(B)** :  $D$  veut  $D(G(z)) \rightarrow 0$  (image fausse)  $\Rightarrow$  maximise  $\log(1 - D(G(z)))$ .
- $G$  veut le contraire : forcer  $D(G(z)) \rightarrow 1$ , donc **minimise (B)** ;
- **Équilibre de Nash** :  $p_g = p_{\text{data}} \Rightarrow D^*(x) = \frac{1}{2}$  partout.

## Analogie

Un **faussaire** ( $G$ ) fabrique de faux billets, un **expert** ( $D$ ) les inspecte. À l'équilibre, les faux sont indiscernables des vrais :  $D(x) = \frac{1}{2}$  — l'expert est réduit au hasard.

# GAN — discriminateur optimal & divergence de Jensen-Shannon

## Discriminateur optimal à $G$ fixé

En dérivant  $V$  point par point ( $\partial_D [p_{\text{data}} \log D + p_g \log(1 - D)] = 0$ ), le maximum est atteint en :

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \quad \implies \quad D^* = \frac{1}{2} \iff p_g = p_{\text{data}}.$$

## Que minimise alors $G$ ?

En substituant  $D^*$  dans  $V$ , on montre que  $G$  minimise en réalité la divergence entre  $p_{\text{data}}$  et  $p_g$  :

$$C(G) = -\log 4 + 2 \underbrace{\text{JSD}(p_{\text{data}} \| p_g)}_{\geq 0, =0 \iff p_g = p_{\text{data}}}$$

La **divergence de Jensen-Shannon (JSD)** est une mesure symétrique de l'écart entre deux distributions — toujours  $\geq 0$ , bornée entre 0 et  $\log 2$ , nulle uniquement si les deux distributions sont identiques.

**Minimum global** :  $C(G^*) = -\log 4$ , atteint uniquement quand  $G$  reproduit exactement  $p_{\text{data}}$ .

## Pathologie 1 : gradients évanescents

Si  $D$  devient trop bon ( $D(G(z)) \approx 0$ ),  $\log(1 - D(G(z)))$  **sature** :  $G$  ne reçoit plus de gradient (exemple chiffré à la slide suivante).

## Pathologie 2 : mode collapse

$G$  se réfugie sur **quelques modes** de  $p_{\text{data}}$  (toujours la même image plausible) : diversité effondrée — motivation du WGAN.

# GAN — loss saturante vs non saturante : exemple chiffré

## Astuce non saturante (Goodfellow et al., 2014)

En pratique  $G$  ne minimise pas  $\log(1 - D(G(z)))$  mais **maximise**  $\log D(G(z))$  — même point fixe, gradients opposés en début d'entraînement. Avec  $D = \sigma(s)$  ( $s$  : logit), les gradients par rapport à  $s$  valent :

$$\frac{\partial}{\partial s} \log(1 - \sigma(s)) = -\sigma(s), \quad \frac{\partial}{\partial s} \log \sigma(s) = 1 - \sigma(s).$$

## Exemple numérique — un pas d'update

Logits du discriminateur :  $s_{\text{réel}} = 2.2$ ,  $s_{\text{faux}} = -0.85$  :

$$D(x) = \sigma(2.2) = 0.900, \quad D(G(z)) = \sigma(-0.85) = 0.299$$

$$\begin{aligned} \mathcal{L}_D &= -[\log 0.900 + \log(1 - 0.299)] \\ &= 0.105 + 0.355 = \mathbf{0.460} \text{ nats.} \end{aligned}$$

Gradient reçu par  $G$  (via  $s_{\text{faux}}$ ) :

$$\text{saturante : } -\sigma(-0.85) = -0.299$$

$$\text{non sat. : } 1 - \sigma(-0.85) = \mathbf{+0.701}.$$

## Début d'entraînement : $D(G(z)) = 0.02$

$G$  produit du bruit,  $D$  le repère facilement :

$$\text{saturante : } -0.02 \quad (\text{évanescent !})$$

$$\text{non sat. : } 1 - 0.02 = \mathbf{0.98} \quad (\text{fort signal})$$

C'est **quand  $G$  est mauvais** que la version non saturante envoie le plus de gradient — exactement ce qu'il faut pour démarrer.

## En PyTorch

La loss non saturante de  $G$  = `BCEWithLogitsLoss` avec les **labels « réels »** sur les images fausses (slide suivante).

# GAN — boucle d'entraînement PyTorch

```
bce = nn.BCEWithLogitsLoss()
for x_real, _ in loader:
    # --- 1) Update du DISCRIMINATEUR ---
    z = torch.randn(b, 100, device=dev)
    x_fake = G(z)
    d_real = D(x_real)
    d_fake = D(x_fake.detach()) # PAS de grad vers G
    loss_D = bce(d_real, torch.ones_like(d_real)) \
        + bce(d_fake, torch.zeros_like(d_fake))
    opt_D.zero_grad(); loss_D.backward(); opt_D.step()

    # --- 2) Update du GENERATEUR ---
    d_fake = D(x_fake) # graphe complet z->G->D
    loss_G = bce(d_fake, torch.ones_like(d_fake))
    opt_G.zero_grad(); loss_G.backward(); opt_G.step()
```

## Les 3 points critiques

1. `x_fake.detach()` dans l'update de  $D$  : sinon le `backward` de  $\mathcal{L}_D$  remonterait dans  $G$  ;
2. **deux `backward()` distincts**, deux optimiseurs (`opt_D`, `opt_G`) ;
3. labels « réels » (`ones`) sur les fausses images pour  $\mathcal{L}_G$  : c'est la loss **non saturante**.

## Hyperparamètres robustes

`Adam(lr=2e-4, betas=(0.5, 0.999))` pour  $G$  et  $D$  (DCGAN) ; alternance stricte 1 update  $D$  / 1 update  $G$ .

# DCGAN — règles architecturales convolutives

Pipeline du générateur (Radford et al., 2016) —  $z \in \mathbb{R}^{100}$  vers une image  $64 \times 64 \times 3$

$z(100) \xrightarrow{\text{ConvTranspose2d}} 4 \times 4 \times 512 \xrightarrow{k4, s2} 8 \times 8 \times 256 \xrightarrow{k4, s2} 16 \times 16 \times 128 \xrightarrow{k4, s2} 32 \times 32 \times 64 \xrightarrow{k4, s2, \tanh} 64 \times 64 \times 3$

## Les règles DCGAN

| Élément         | Générateur                                 | Discriminateur                                         |
|-----------------|--------------------------------------------|--------------------------------------------------------|
| Échantillonnage | <code>ConvTranspose2d</code><br>$k=4, s=2$ | <code>Conv2d</code> $s=2$ (pas de pooling)             |
| Normalisation   | <code>BatchNorm2d</code>                   | <code>BatchNorm2d</code> (sauf 1 <sup>re</sup> couche) |
| Activation      | <code>ReLU</code>                          | <code>LeakyReLU(0.2)</code>                            |
| Sortie          | $\tanh([-1, 1])$                           | logit (1 scalaire)                                     |
| Couches denses  | aucune                                     | aucune                                                 |

## Pourquoi ces choix ?

- $\tanh$  en sortie de  $G \Leftrightarrow$  données normalisées dans  $[-1, 1]$  (comme la diffusion, TP 09) ;
- `LeakyReLU` dans  $D$  : un gradient subsiste même pour les activations négatives —  $G$  en dépend ;
- $k=4, s=2$  :  $k$  multiple de  $s \Rightarrow$  pas d'artefacts en damier (cf. Module 3) ;
- `BatchNorm` stabilise le jeu adversarial (évite l'effondrement des statistiques).



# WGAN — distance de Wasserstein & gradient penalty

## Changer de distance (Arjovsky et al., 2017)

La JSD **sature** quand les supports de  $p_{\text{data}}$  et  $p_g$  sont disjoints (cas typique en haute dimension) : gradient nul,  $G$  ne progresse plus. La **distance de Wasserstein-1** ("Earth Mover Distance") mesure le coût minimal pour déplacer la masse de  $p_g$  vers  $p_{\text{data}}$  — elle reste informative même quand les supports sont disjoints.

Par la **dualité de Kantorovich-Rubinstein** :

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \leq 1} \underbrace{\mathbb{E}_{p_{\text{data}}}[f(x)]}_{\text{score moyen sur réel}} - \underbrace{\mathbb{E}_{p_g}[f(\tilde{x})]}_{\text{score moyen sur généré}}$$

Le « discriminateur » devient un **critique**  $f$  : sortie réelle (pas de sigmoïde), contrainte **1-Lipschitz** ( $|f(a) - f(b)| \leq \|a - b\|$ ).

## Imposer la contrainte de Lipschitz

- **Weight clipping** (WGAN) : `p.data.clamp_(-0.01, 0.01)` + **RMSprop** — simple mais brutal (capacité bridée) ;
- **Gradient penalty** (WGAN-GP, Gulrajani et al., 2017) : pénaliser  $\|\nabla\| \neq 1$  sur des interpolations  $\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}$  :

$$\mathcal{L}_{\text{GP}} = \lambda \mathbb{E}_{\hat{x}} \left[ \left( \|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1 \right)^2 \right], \quad \lambda=10.$$

## Ce que l'on y gagne

- la loss du critique **corrèle avec la qualité visuelle**  $\Rightarrow$  courbe d'entraînement enfin interprétable ;
- gradients utiles même si les distributions sont disjointes  $\Rightarrow$  entraînement **nettement plus stable** ;
- **mode collapse** fortement réduit ;
- coût : plusieurs updates du critique par update de  $G$  ( $n_{\text{critic}} = 5$ ).

# cGAN — génération conditionnelle

## Conditionner le jeu par une variable $y$ (Mirza & Osindero, 2014)

Classe, attribut, ou même une image entière —  $y$  est fourni **aux deux joueurs** :

$$\min_G \max_D \mathbb{E}_{x,y} [\log D(x|y)] + \mathbb{E}_{z,y} [\log (1 - D(G(z|y)))] .$$

$D$  ne juge plus « est-ce réel ? » mais **« est-ce réel et cohérent avec  $y$  ? »**.

## Implémentation PyTorch

- `label → nn.Embedding(n_classes, d);`
- dans  $G$  : embedding **concaténé à  $z$**  ;
- dans  $D$  : embedding projeté en carte spatiale et **concaténé aux canaux de  $x$**  ;
- même boucle d'entraînement, mêmes pertes.

## Cas d'usage

- génération **par classe** : équilibrer un dataset médical (lésions rares) ;
- synthèse contrôlée pour tests : « générer un défaut industriel de type  $k$  » ;
- quand  $y$  est une **image** : on obtient la traduction d'image → **Pix2Pix** (slide suivante).

## Déjà vu au Module 3

Même principe que le **CVAE** (TP 08) et la diffusion conditionnelle — le conditionnement par embedding est un motif transversal du génératif.

# Pix2Pix — traduction d'image avec PatchGAN

Objectif hybride (Isola et al., 2017) : adversarial + fidélité  $L_1$

Paires alignées  $(x, y)$  (ex. IRM  $\rightarrow$  CT). Le générateur (un **U-Net**, Module 3!) est entraîné par :

$$G^* = \arg \min_G \max_D \underbrace{\mathbb{E}[\log D(x, y)] + \mathbb{E}[\log(1 - D(x, G(x)))]}_{\mathcal{L}_{\text{cGAN}} : \text{hautes fréquences (textures)}} + \lambda \underbrace{\mathbb{E}[\|y - G(x)\|_1]}_{\mathcal{L}_{L_1} : \text{basses fréquences}}, \quad \lambda = 100.$$

## PatchGAN : un discriminateur markovien

$D$  ne rend pas **un** verdict global mais une **carte** de verdicts, chacun ne voyant qu'un patch  $70 \times 70$  (champ réceptif d'un petit CNN convolutif) ; la loss moyenne sur les patches.

- hypothèse : le **style/texture est local** — juger des patches suffit ;
- moins de paramètres, applicable à toute taille d'image ;
- la  $L_1$  s'occupe de la **cohérence globale** que PatchGAN ne voit pas.

## Applications de traduction

- IRM  $\rightarrow$  CT-scan (planification radiothérapie) ;
- cartes  $\rightarrow$  photos satellites, plans  $\rightarrow$  façades ;
- colorisation, inpainting, restauration.

## Sans paires alignées ?

**CycleGAN** : deux GAN couplés  $G : X \rightarrow Y$ ,  $F : Y \rightarrow X$  + contrainte de **cycle-consistance**  
 $\mathbb{E}\|F(G(x)) - x\|_1 + \mathbb{E}\|G(F(y)) - y\|_1$  — traduction de domaine non supervisée.

# SRGAN — super-résolution perceptuelle

## Le problème du MSE seul (Ledig et al., 2017)

Optimiser le PSNR (MSE pixel) produit des images **lisses et floues** : les hautes fréquences plausibles sont “moyennées”. SRGAN remplace la fidélité pixel par une **loss perceptuelle** à deux termes :

$$\ell^{SR} = \underbrace{\left\| \phi_k(I^{HR}) - \phi_k(G(I^{LR})) \right\|_F^2}_{\text{content loss VGG: écart dans l'espace de features de } \phi_k} + 10^{-3} \underbrace{\mathcal{L}_{\text{adv}}(G)}_{\text{loss adversariale}}$$

**Terme 1** :  $\phi_k$  = activations d'un **VGG19 pré-entraîné et gelé** — on compare des **percepts** (textures, structures), pas des pixels. **Terme 2** : force  $G$  à produire des images réalistes.

## Architecture

- $G$  : **ResNet** ( $B=16$  blocs résiduels) + upsampling **sub-pixel nn.PixelShuffle(2)** ( $\times 4$  au total) ;
- $D$  : CNN VGG-style (ou **PatchGAN**) ;
- entraînement en 2 temps : pré-entraînement MSE (SRResNet), puis fine-tuning adversarial.

## Applications

- super-résolution d'images **médicales** (CT basse dose, IRM rapide) ;
- imagerie **satellite** (Sentinel-2) et vidéosurveillance ;
- compromis : PSNR  $\downarrow$  mais réalisme perçu  $\uparrow$  — d'où l'importance des métriques perceptuelles (slide suivante).

# Métriques des modèles génératifs : PSNR, SSIM, IS, FID

## Fidélité à une référence (reconstruction, SR)

**PSNR** — rapport signal/bruit (en dB), **plus haut = mieux** :

$$\text{PSNR} = 10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}} \xrightarrow[\text{MAX}=1]{\text{MSE}=10^{-3}} 30 \text{ dB}$$

**SSIM** — compare ce que l'œil perçoit, comme **produit de 3 facteurs**  $\in [0, 1]$  :

$$\text{SSIM} = \underbrace{l(x, y)}_{\text{luminance}} \cdot \underbrace{c(x, y)}_{\text{contraste}} \cdot \underbrace{s(x, y)}_{\text{structure}}$$

SSIM = 1 si images identiques — bien plus proche du jugement humain que le PSNR.

## Qualité d'une distribution générée (GAN, diffusion)

**FID** — distance entre les **features Inception-v3** des images réelles ( $r$ ) et générées ( $g$ ), **plus bas = mieux** :

$$\text{FID} = \underbrace{\|\mu_r - \mu_g\|^2}_{\text{écart des moyennes}} + \underbrace{\text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})}_{\text{écart des covariances}}$$

Sensible à la qualité et à la diversité : le mode collapse fait **exploser** le FID.

**IS (Inception Score)** : mesure si les images sont nettes et variées — historique, supplanté par le FID (qui compare au réel).

## Outils PyTorch

| Métrique    | Outil                                                                               | Quand l'utiliser                                           |
|-------------|-------------------------------------------------------------------------------------|------------------------------------------------------------|
| PSNR / SSIM | <code>torchmetrics</code> ( <code>PeakSignalNoiseRatio</code> , <code>SSIM</code> ) | débruitage, super-résolution, traduction alignée           |
| FID         | <code>pytorch-fid</code> , <code>torchmetrics.image.fid</code>                      | GAN / diffusion : qualité + diversité ( $\geq 10k$ images) |
| IS          | <code>torchmetrics.image.inception</code>                                           | historique ; préférer le FID (compare au réel)             |

# GAN — cas d'usage en Computer Vision

## Quelle variante pour quel problème ?

| Cas d'usage               | Variante                    | Métriques           | Exemple concret                      |
|---------------------------|-----------------------------|---------------------|--------------------------------------|
| Synthèse pure             | DCGAN, WGAN-GP              | FID, IS             | visages, textures, augmentation      |
| Augmentation médicale     | cGAN (par classe)           | FID + macro-F1 aval | lésions dermato rares (IRM, dermato) |
| Traduction alignée        | Pix2Pix (U-Net + Patch-GAN) | SSIM, FID           | IRM → CT, plans → photos             |
| Traduction non alignée    | CycleGAN                    | FID                 | domaine A ↔ B sans paires            |
| Super-résolution          | SRGAN / ESRGAN              | PSNR, SSIM, FID     | CT basse dose, satellite, vidéo      |
| Restauration / inpainting | Pix2Pix, GAN + masque       | PSNR, SSIM          | images dégradées, artefacts          |

Rappel du Module 3 : la diffusion dépasse les GAN en qualité/diversité mais reste plus lente à l'échantillonnage — le GAN garde l'avantage de la génération **en une passe** (temps réel).

# Vision Transformers : ViT, Swin & SegFormer

---

# De la convolution à l'attention

## Ce que « sait » un CNN (biais inductifs)

- **localité** : un pixel interagit avec ses voisins ;
- **invariance par translation** : mêmes poids partout ;
- champ réceptif **croissant avec la profondeur** — l'information lointaine met des couches à se rejoindre.

Ces a priori sont une **force** quand les données sont rares... et une **limite** quand elles abondent.

## Le pari du Transformer (Dosovitskiy et al., 2021)

- **aucun** biais spatial : l'image est une **séquence de patches** ;
- chaque patch peut attendre sur **tous** les autres dès la couche 1 (champ réceptif global immédiat) ;
- les relations utiles sont **appprises**, pas imposées — à condition d'avoir assez de données (ou un pré-entraînement massif).

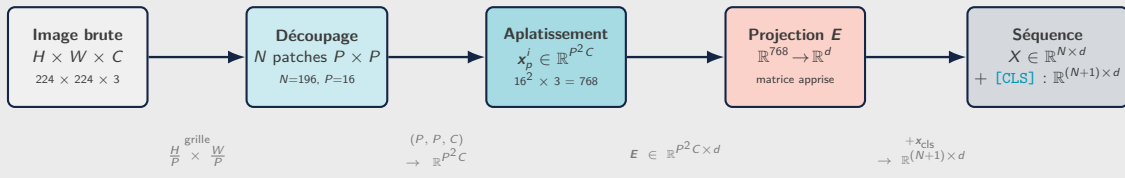
## Le message empirique

À petit dataset : CNN  $>$  ViT (les biais inductifs compensent). Avec un grand pré-entraînement (ImageNet-21k, JFT) : ViT  $\geq$  CNN, et **le fine-tuning d'un ViT pré-entraîné** devient le choix par défaut — exactement le réflexe transfer learning du Module 2.



# ViT — du pixel au token : le patch embedding

L'entrée du ViT n'est pas une image brute — c'est une séquence de vecteurs d'embedding



Calculs concrets — ViT-Base/16, image 224 × 224 × 3

$$N = \frac{H}{P} \times \frac{W}{P} = \frac{224}{16} \times \frac{224}{16} = \mathbf{196 \text{ patches}}$$

$$\text{dim. patch aplati} = P^2 \cdot C = 16^2 \times 3 = \mathbf{768}$$

$$\mathbf{X} \in \mathbb{R}^{196 \times d} \xrightarrow{+[\text{CLS}]} \mathbf{X} \in \mathbb{R}^{197 \times d}$$

Analogie NLP : le patch comme un mot

En NLP, un Transformer reçoit une séquence de vecteurs de mots. Le ViT fait la même chose avec des **vecteurs de patches**. La matrice  $E$  joue le rôle de la table d'embedding : elle apprend quelle information de chaque région est pertinente.

Point clé

$\mathbf{X} \in \mathbb{R}^{N \times d}$  n'est **pas** une matrice de pixels bruts : c'est une **séquence de vecteurs d'embedding de patches**, permettant d'appliquer l'attention exactement comme en NLP.

# Self-attention — Q, K, V : le mécanisme

## Intuition : trois rôles par token

Chaque token joue **trois rôles** simultanément :

- **Q** (*Query*)  
"que cherche-je ?"  
— vecteur de recherche ;
- **K** (*Key*)  
"de quoi parle-je ?"  
— étiquette du token ;
- **V** (*Value*)  
"quelle info je fournis ?"  
— contenu à transmettre.

## Pourquoi $\sqrt{d_k}$ ?

Si  $\mathbf{q}, \mathbf{k} \sim \mathcal{N}(0, 1)$  composante à composante,  $\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$  : les scores croissent avec  $d_k$ , le softmax **sature** (gradients  $\approx 0$ ). Diviser par  $\sqrt{d_k}$  ramène la variance à 1.

## Scaled dot-product attention (Vaswani et al., 2017)

Pour  $\mathbf{X} \in \mathbb{R}^{n \times d}$ , trois projections apprises ( $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d_k}$ ) :

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)}_{\text{poids } \mathbf{A} \in \mathbb{R}^{n \times n}} \mathbf{V}$$

Lecture étape par étape :

1.  $\mathbf{Q}\mathbf{K}^\top$  : matrice  $n \times n$  des scores —  $\text{score}(i, j) = \mathbf{q}_i \cdot \mathbf{k}_j$  mesure l'affinité entre tokens  $i$  et  $j$  ;
2.  $\div \sqrt{d_k}$  : évite la saturation du softmax quand  $d_k$  est grand (voir ci-dessous) ;
3. softmax (par ligne) : transforme les scores en poids  $\geq 0$  qui somment à 1 ;
4.  $\times \mathbf{V}$  : chaque token reçoit une **moyenne pondérée** du contenu de tous les tokens.

## Coût quadratique en $n$

$\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times n}$  : coût  $\mathcal{O}(n^2 d_k)$ . Pour  $n=196$  (ViT-Base/16) — raisonnable. En haute résolution ( $n \gg 1000$ )  $\Rightarrow$  **Swin** (fenêtres) ou **SegFormer** (réduction).

# Self-attention — exemple numérique complet

Mini-séquence :  $n = 2$  tokens,  $d_k = 2$

$$q_1 = (1, 0), \quad k_1 = (1, 0), \quad k_2 = (0, 1), \quad v_1 = (1, 0), \quad v_2 = (0, 2).$$

1) Scores (produit scalaire  $\div \sqrt{d_k} = \sqrt{2}$ ) :

$$s_{11} = \frac{q_1 \cdot k_1}{\sqrt{2}} = 0.707, \quad s_{12} = \frac{q_1 \cdot k_2}{\sqrt{2}} = 0.$$

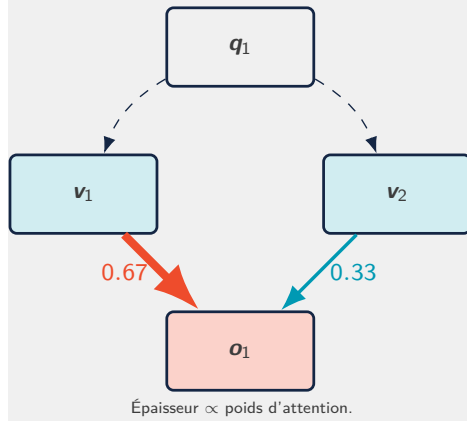
2) Softmax (scores  $\rightarrow$  poids qui somment à 1) :

$$a_{11} = \frac{e^{0.707}}{e^{0.707} + e^0} = \mathbf{0.670}, \quad a_{12} = \mathbf{0.330}.$$

3) Sortie (moyenne pondérée des valeurs) :

$$o_1 = 0.670 (1, 0) + 0.330 (0, 2) = \mathbf{(0.670, 0.660)}.$$

## Routage de l'information



## Idée clé

Poids dépendant du **contenu** (vs convolution à poids fixes) :  
un **routage adaptatif**.

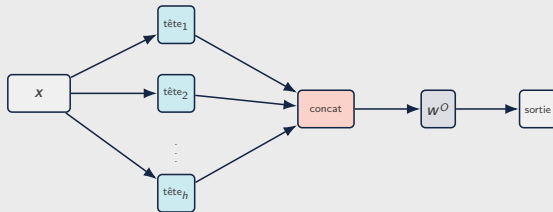
# Multi-Head Attention

## $h$ têtes en parallèle

Une seule attention = un seul « type de relation ». On projette vers  $h$  sous-espaces de dimension  $d_k = d/h$ , chaque tête fait son attention, puis on **concatène** et on reprojette :

$$\text{head}_i = \text{Attention}(xW_i^Q, xW_i^K, xW_i^V)$$

$$\text{MHA}(\mathbf{X}) = [\text{head}_1; \dots; \text{head}_h] W^O$$



Même coût total qu'une grosse tête, mais  $h$  **relations apprises indépendamment** (texture, contours, position...).

```
import torch.nn as nn
```

```
mha = nn.MultiheadAttention(  
    embed_dim=768, num_heads=12,  
    batch_first=True)
```

```
x = torch.randn(8, 197, 768)  # (B, n, d)  
out, attn = mha(x, x, x,      # self-attention  
                need_weights=True)  
print(out.shape)  # (8, 197, 768)  
print(attn.shape) # (8, 197, 197)
```

## À noter

`attn` est la carte d'attention moyenne — celle que l'on visualise pour l'**interprétabilité** (attention maps, cf. XAI du Module 2).

## ViT-Base

$d = 768, h = 12 \Rightarrow d_k = 64$  par tête.

# ViT — les équations du Transformer, terme à terme

## Étape 0 — construction de l'entrée $\mathbf{z}_0 \in \mathbb{R}^{(N+1) \times d}$

$$\mathbf{z}_0 = \left[ \underbrace{\mathbf{x}_{\text{cls}}}_{\substack{\text{token [CLS]} \\ \text{appris, } \mathbb{R}^d}} ; \underbrace{\mathbf{x}_p^1 \mathbf{E}}_{\substack{\text{patch 1 projeté}}}; \dots; \underbrace{\mathbf{x}_p^N \mathbf{E}}_{\substack{\text{patch } N \text{ projeté}}} \right] + \underbrace{\mathbf{E}_{\text{pos}}}_{\substack{\text{encodage de} \\ \text{position appris}}}$$

- $\mathbf{E} \in \mathbb{R}^{P^2 C \times d}$  : matrice de projection des patches, apprise ( $P^2 C = 768$  pour ViT-Base/16) ;
- $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times d}$  : encodage de position appris — sans lui les patches seraient un ensemble non ordonné ;
- $\mathbf{x}_{\text{cls}} \in \mathbb{R}^d$  : token spécial ajouté en tête (comme BERT) qui agrège l'information globale.

## Étapes 1 à $L$ — blocs Transformer (pré-norm)

Pour  $\ell = 1, \dots, L$  :

$$\begin{aligned} \mathbf{z}'_{\ell} &= \underbrace{\text{MHA}}_{\text{self-attention}} \left( \underbrace{\text{LN}(\mathbf{z}_{\ell-1})}_{\text{norm. pré-bloc}} \right) + \underbrace{\mathbf{z}_{\ell-1}}_{\text{résidu}} \\ \mathbf{z}_{\ell} &= \underbrace{\text{MLP}}_{\text{2 couches GELU}} \left( \text{LN}(\mathbf{z}'_{\ell}) \right) + \mathbf{z}'_{\ell} \end{aligned}$$

Les **connexions résiduelles** (“ $+\mathbf{z}_{\ell-1}$ ”) permettent le gradient de passer directement à travers  $L$  blocs, comme en ResNet.

## Étape finale — prédiction

$$\hat{y} = \text{LN} \left( \underbrace{\mathbf{z}_L^0}_{\substack{\text{token [CLS]} \\ \text{à la couche } L}} \right) \mathbf{w}_{\text{head}}$$

Seul le token [CLS] (indice 0) est utilisé pour la classification — les  $N$  tokens restants sont ignorés en sortie.

## ViT-Base/16 en chiffres

$L=12$  blocs,  $d=768$ ,  $h=12$  têtes  
 $d_k=64$  par tête, MLP expansion  $4\times$   
 $197=196+1$  tokens en entrée

**86M paramètres**

`nn.TransformerEncoderLayer(norm_first=True)`

# Swin Transformer — attention par fenêtres décalées

## Le problème du ViT en haute résolution

L'attention globale coûte  $\mathcal{O}(n^2)$  en nombre de patches : impraticable en haute résolution. Swin (Liu et al., 2021) la restreint à des **fenêtres locales** de taille fixe  $M \times M$  ( $M=7$ ).

**Gain de complexité** (pour  $h \times w$  patches de dimension  $C$ , terme dominant) :

$$\underbrace{2(hw)^2 C}_{\text{attention globale, quadratique en } hw} \xrightarrow{\text{Swin}} \underbrace{2M^2 hw C}_{\text{attention fenêtrée, linéaire en } hw}$$

Le facteur  $(hw)^2$  est remplacé par  $M^2 \cdot hw$  : puisque  $M=7$  est fixe, le coût devient **linéaire** en  $hw$ .

## Exemple chiffré (stage 1 : $56 \times 56$ patches, $C=96$ )

Terme d'attention :  $2(hw)^2 C = 2 \cdot 3136^2 \cdot 96 \approx 1.9 \cdot 10^{12}$

Fenêtré :  $2M^2 hw C = 2 \cdot 49 \cdot 3136 \cdot 96 \approx 3.0 \cdot 10^7$

$\Rightarrow$  rapport =  $(hw)/M^2 = 3136/49 : 64 \times$  **moins de calcul**.

## Fenêtres décalées (SW-MSA)

Des fenêtres fixes ne communiquent jamais entre elles. Un bloc sur deux **décale** la grille de  $\lfloor M/2 \rfloor$  : les frontières changent, l'information **circule entre fenêtres** (implémenté par décalage cyclique `torch.roll` + masques).

## Hiérarchie à la CNN

- 4 étages : résolutions  $\frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}$  (**patch merging**  $2 \times 2 \rightarrow$  canaux  $\times 2$ ) ;
- produit des **cartes multi-échelles** — réutilisables par FPN, U-Net, têtes de détection/segmentation ;
- biais de position **relatif** appris  $B$  ajouté aux scores :  $\text{softmax}(QK^T / \sqrt{d} + B)V$  ;
- le ViT « backbone généraliste » : classification, détection, segmentation.

# SegFormer — segmentation sémantique efficace

## Encodeur MiT (Mix Transformer)

- hiérarchique (4 étages, comme Swin) ;
- **attention à séquence réduite** :  $K$ ,  $V$  sous-échantillonnés d'un facteur  $R \Rightarrow$  coût  $\mathcal{O}(n^2/R)$  ;
- **pas de positional encoding** : une **Conv**  $3 \times 3$  dans le MLP (Mix-FFN) capture la position  $\Rightarrow$  robuste aux changements de résolution ;
- décodeur **tout-MLP** : fusion des 4 échelles, léger et rapide.

## Métrique de segmentation

$$\text{mIoU} = \frac{1}{K} \sum_{k=1}^K \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k + \text{FN}_k}$$

moyenne sur les  $K$  classes de l'intersection sur union (cf. [torchmetrics.JaccardIndex](#)).

```
from transformers import (
    SegformerForSemanticSegmentation,
    SegformerImageProcessor)

name = ("nvidia/segformer-b2-"
        "finetuned-ade-512-512")
proc = SegformerImageProcessor.from_pretrained(name)
model = SegformerForSemanticSegmentation \
        .from_pretrained(name)

inputs = proc(images=img, return_tensors="pt")
logits = model(**inputs).logits # (B,K,H/4,W/4)
mask = logits.argmax(dim=1)     # classes par pixel
```

## Applications

Segmentation d'organes/tumeurs (IRM, CT), rétine ; scènes urbaines (CityScapes) ; bâtiments et routes (satellite).

# CNN vs ViT — quel modèle pour quelles données ?

## Comparaison synthétique

| Critère                              | CNN (ResNet, EfficientNet)      | ViT / Swin                                       |
|--------------------------------------|---------------------------------|--------------------------------------------------|
| Biais inductifs                      | localité, translation (forts)   | quasi aucun (appris)                             |
| Données nécessaires                  | modestes (qq. milliers)         | grandes, ou <b>pré-entraînement</b> massif       |
| Champ réceptif                       | local, croît avec la profondeur | <b>global dès la 1<sup>re</sup> couche</b>       |
| Coût en résolution                   | linéaire en pixels              | quadratique (ViT) → linéaire (Swin)              |
| Interprétabilité                     | Grad-CAM (Module 2)             | <b>attention maps</b> natives                    |
| Robustesse aux occlusions / textures | sensible à la texture           | plus orienté <b>forme</b> , souvent plus robuste |

## Règle pratique pour la formation

Dataset < 10k images, pas de pré-entraînement adapté ⇒ **CNN fine-tuné** (Module 2). Pré-entraînement disponible (ImageNet-21k, DINO) ou tâche dense en haute résolution ⇒ **ViT/Swin fine-tuné via timm**, SegFormer pour la segmentation. Dans le doute : **benchmarker les deux** — c'est l'un des sujets du projet fil rouge.



# Écosystème pratique : timm & Hugging Face

```
import timm

# ViT pre-entraîne, tête remplacée (7 classes)
model = timm.create_model(
    "vit_base_patch16_224",
    pretrained=True, num_classes=7)

# Swin pour la classification fine-grained
swin = timm.create_model(
    "swin_base_patch4_window7_224",
    pretrained=True, num_classes=7)

# Fine-tuning : memes recettes qu'au Module 2
# (gel des couches, lr différentiel, OneCycleLR)
for p in model.blocks[:8].parameters():
    p.requires_grad = False
```

## Ce que fournit l'écosystème

- **timm** : des centaines de backbones (ViT, Swin, DeiT, EfficientNet) pré-entraînés, API uniforme ;
- **transformers** (HF) : ViT, SegFormer, DINO + processors d'entrée ;
- **einops** : `rearrange(x, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)')` — le patch embedding en une ligne lisible.

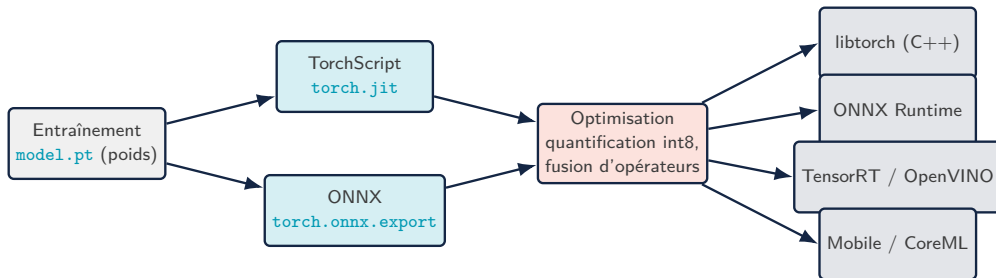
## Interprétabilité : attention maps

Visualiser l'attention du [CLS] vers les patches (moyenne des têtes, *attention rollout*) ; spectaculaire avec **DINO** (ViT auto-supervisé) : les cartes segmentent les objets **sans aucun label**. Complète Grad-CAM/SHAP du Module 2 pour l'audit des modèles.

## Déploiement : TorchScript, ONNX & quantification

---

# Du notebook à la production — vue d'ensemble



## Pourquoi exporter ? Le checkpoint .pt ne suffit pas

Un `state_dict` exige **Python + la classe du modèle** pour être rechargé. La production veut un artefact **autonome** : exécutable sans Python (serveur C++, GPU embarqué, mobile), au **graphe figé** (optimisable, versionnable, auditable) et **rapide** (fusion d'opérateurs, précision réduite).

## Deux familles d'export

**TorchScript** : reste dans l'écosystème PyTorch (libtorch). **ONNX** : format **ouvert** interopérable (TensorRT, OpenVINO, CoreML, navigateur).

## Règle d'or

Toujours **valider numériquement** l'export : mêmes entrées  $\Rightarrow$  sorties identiques (à  $10^{-5}$  près) entre le modèle Python et l'artefact exporté.

# TorchScript — `trace` vs `script`

```
model.eval()
x = torch.randn(1, 3, 224, 224)

# 1) TRACE : enregistre les ops exécutées
traced = torch.jit.trace(model, x)

# 2) SCRIPT : compile un sous-ensemble Python
scripted = torch.jit.script(model)

traced.save("model_traced.pt")
m = torch.jit.load("model_traced.pt")

# validation numérique de l'export
assert torch.allclose(model(x), m(x),
                       atol=1e-5)
```

## Quelle méthode choisir ?

|                                          | <code>trace</code>             | <code>script</code>     |
|------------------------------------------|--------------------------------|-------------------------|
| Principe                                 | exécute et enregistre          | compile le code         |
| Contrôle de flux ( <code>if/for</code> ) | <b>figé</b> sur l'exemple      | préservé                |
| Limites                                  | branches dépendant des données | sous-ensemble de Python |

## Piège classique

Un `if x.mean() > 0:` dans le `forward` est **silencieusement figé** par `trace` (seule la branche prise à l'export survit) — utiliser `script`, ou vérifier avec des entrées couvrant les deux branches.

## Et ensuite ?

L'artefact `.pt` se charge en **C++** (`torch::jit::load`) — aucun interpréteur Python en production.

# ONNX — le format d'échange interopérable

```
torch.onnx.export(  
    model, x, "model.onnx",  
    opset_version=17,  
    input_names=["image"],  
    output_names=["logits"],  
    dynamic_axes={"image": {0: "batch"},  
                  "logits": {0: "batch"}})  
  
import onnxruntime as ort  
sess = ort.InferenceSession(  
    "model.onnx",  
    providers=["CPUExecutionProvider"])  
out = sess.run(None,  
    {"image": x.numpy()})[0]  
  
assert np.allclose(out,  
    model(x).detach().numpy(), atol=1e-5)
```

## Points clés de l'export

- `opset_version` : version du jeu d'opérateurs ONNX (fixe la compatibilité runtime) ;
- `dynamic_axes` : batch variable à l'inférence (sinon la taille de l'exemple est **figée**) ;
- l'export **trace** le modèle : mêmes précautions de contrôle de flux que `jit.trace`.

## Runtimes cibles

**ONNX Runtime** (CPU/GPU, serveur & edge), **TensorRT** (GPU NVIDIA, fusion + FP16/INT8), **OpenVINO** (CPU Intel), **CoreML** (Apple). Un seul artefact, plusieurs cibles matérielles.

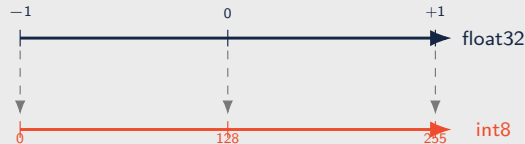
# Quantification & PyTorch Mobile

## Principe : remplacer chaque float32 par un entier int8 (sur 256 niveaux)

On encode un réel  $x$  par un entier  $q \in [0, 255]$ , puis on le décode :

$$q = \left\lfloor \frac{x}{s} \right\rfloor + z \quad \hat{x} = s(q - z)$$

- $s = \text{échelle}$  (taille d'un pas) =  $\frac{x_{\max} - x_{\min}}{255}$  ;
- $z = \text{zéro-point}$  (l'entier qui code  $x=0$ ) ;
- $\lfloor \cdot \rfloor = \text{arrondi}$  ;  $\hat{x} = \text{valeur reconstruite}$  ( $\approx x$ ).



$$s = \frac{2}{255} \approx 0.008, \quad z = 128$$

Erreur max de quantification =  $s/2 \approx 0.004$  — négligeable pour la plupart des classifieurs.

## Trois stratégies

- **dynamique** : poids int8, activations quantifiées à la volée — 1 ligne, idéal Linear/LSTM ;
- **statique** (PTQ) : calibration sur quelques batches → échelles d'activations figées ;
- **QAT** : fausse quantification pendant le fine-tuning — la meilleure précision finale.

```
qmodel = torch.quantization.quantize_dynamic(  
    model, {nn.Linear}, dtype=torch.qint8)
```

## Gains typiques (CPU)

| Critère       | float32    | int8                                |
|---------------|------------|-------------------------------------|
| Taille modèle | $\times 1$ | $\approx \times 1/4$                |
| Latence CPU   | $\times 1$ | $\times 2$ à $\times 4$ plus rapide |
| Précision     | référence  | -0 à -1 pt (PTQ)                    |

## PyTorch Mobile

`torch.utils.mobile_optimizer.optimize_for_mobile(scripted)` : fusion  
Conv+BN+ReLU, nettoyage du graphe → artefact `.pt1` pour **iOS / Android** — à combiner  
avec la quantification.

## Projet fil rouge & synthèse

---

# Projet fil rouge — organisation & thèmes

## Format

Équipes de **2–3 participants** ; un **pipeline DL complet** sur un cas réel, initié dès le Module 1 et enrichi à chaque module (données → modèle → évaluation → interprétabilité → export).

## Thèmes proposés

| Thème                                                                     | Architectures               | Métriques                |
|---------------------------------------------------------------------------|-----------------------------|--------------------------|
| CV médicale : classification / segmentation (IRM cérébrale, rétinopathie) | CNN, ViT, U-Net / SegFormer | accuracy, F1, mIoU, Dice |
| Contrôle qualité industriel : défauts + localisation                      | CNN + CAM / Grad-CAM        | F1, mAP                  |
| Maintenance prédictive : RUL turbofan (NASA C-MAPSS)                      | LSTM + CNN 1D               | MAE, RMSE                |
| Super-résolution satellite (Sentinel-2)                                   | SRGAN ou diffusion          | PSNR, SSIM, FID          |
| Segmentation urbaine (CityScapes)                                         | SegFormer                   | mIoU                     |
| Génération médicale par diffusion (IRM/CT)                                | DDPM conditionnel           | FID + F1 aval            |
| Audit XAI complet d'un classifieur médical                                | Grad-CAM + SHAP + rapport   | fidélité, plausibilité   |



## Critères d'évaluation

1. **Pertinence du choix architectural** et justification (pourquoi ce modèle pour ces données ?) ;
2. **Qualité du pipeline** : prétraitement, augmentation, validation croisée, reproductibilité (seeds) ;
3. **Performance des métriques** adaptées au cas d'usage (accuracy, mAP, IoU, SSIM... ) ;
4. **Clarté de la présentation** et démo live.

## Livrables attendus

- dépôt de code structuré (données **non commitées**, README, checkpoints nommés) ;
- notebook ou scripts reproductibles de bout en bout ;
- un volet **interprétabilité** (Grad-CAM / SHAP / attention maps) ;
- un **export d'inférence** (TorchScript ou ONNX) validé numériquement ;
- présentation finale avec démo.

## Conseil

Réutiliser les briques des TP : pipelines DermaMNIST, boucles d'entraînement, conventions de checkpoints — le projet **assemble**, il ne repart pas de zéro.

# Synthèse mathématique du Module 4

| Concept            | Formule clé                                                                                                                                                       | PyTorch / outil                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| GAN minimax        | $\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$                                                                           | 2 <code>backward()</code> , <code>detach()</code> |
| $D$ optimal        | $D^* = p_{\text{data}} / (p_{\text{data}} + p_g)$ , $C(G) = 2 \text{ JSD} - \log 4$                                                                               | <code>BCEWithLogitsLoss</code>                    |
| Loss non saturante | $G$ maximise $\log D(G(\mathbf{z}))$                                                                                                                              | labels <code>ones</code> sur les fausses          |
| WGAN-GP            | $\sup_{\ f\ _L \leq 1} \mathbb{E}f(\mathbf{x}) - \mathbb{E}f(\tilde{\mathbf{x}}) + \lambda \mathbb{E}(\ \nabla f\  - 1)^2$                                        | critique sans sigmoïde                            |
| cGAN               | $D(\mathbf{x} \mathbf{y})$ , $G(\mathbf{z} \mathbf{y})$                                                                                                           | <code>nn.Embedding</code> concaténé               |
| Pix2Pix            | $\mathcal{L}_{\text{cGAN}} + \lambda \mathbb{E}\ \mathbf{y} - G(\mathbf{x})\ _1$ , $\lambda=100$                                                                  | U-Net + PatchGAN                                  |
| SRGAN              | loss VGG (features) + $10^{-3}$ adversariale                                                                                                                      | <code>nn.PixelShuffle</code> , VGG gelé           |
| FID                | $\ \boldsymbol{\mu}_r - \boldsymbol{\mu}_g\ ^2 + \text{Tr}(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r \boldsymbol{\Sigma}_g)^{1/2})$ | <code>pytorch-fid</code>                          |
| Self-attention     | $\text{softmax}(\mathbf{QK}^\top / \sqrt{d_k}) \mathbf{V}$                                                                                                        | <code>nn.MultiheadAttention</code>                |
| ViT                | $\mathbf{z}_0 = [\mathbf{x}_{\text{cls}}; \mathbf{x}_p \mathbf{E}] + \mathbf{E}_{\text{pos}}$ , blocs MHA/MLP pré-norm                                            | <code>timm.create_model</code>                    |
| Swin (W-MSA)       | $2M^2 hwC$ au lieu de $2(hw)^2 C$                                                                                                                                 | <code>swin_base_patch4...</code>                  |
| SegFormer / mIoU   | $\text{mIoU} = \frac{1}{K} \sum_k \text{TP}_k / (\text{TP}_k + \text{FP}_k + \text{FN}_k)$                                                                        | <code>transformers</code> (HF)                    |
| Quantification     | $q = \lfloor x/s \rfloor + z$ , $\hat{x} = s(q - z)$                                                                                                              | <code>quantize_dynamic</code>                     |
| Export             | graphe figé, validation $ f(\mathbf{x}) - \hat{f}(\mathbf{x})  < 10^{-5}$                                                                                         | <code>torch.jit</code> / <code>torch.onnx</code>  |

## Trois ouvrages couvrent les concepts présentés

1. **Atienza, R.** (2018). *Advanced Deep Learning with Keras*. Packt Publishing. ISBN 978-1-78862-941-6.  
Chapitres pertinents : GAN (DCGAN, cGAN), WGAN, GAN désenchevêtrés, super-résolution.
2. **Vasilev, I.** (2019). *Python Deep Learning* (2<sup>e</sup> éd.). Packt Publishing.  
Chapitres pertinents : modèles génératifs (GAN), mécanismes d'attention.
3. **Buduma, N., Buduma, N., & Papa, J.** (2022). *Fundamentals of Deep Learning* (2<sup>e</sup> éd.). O'Reilly Media. ISBN 978-1-492-08128-7.  
Chapitres pertinents : modèles génératifs, attention & transformers, mise en pratique.

*Les références couvrent les fondements théoriques de toutes les architectures vues dans la formation (Modules 1 à 4).*

# Générer, transformer, déployer.

**Fin du parcours — place au projet fil rouge !**



bassem.benhamed@enetcom.usf.tn